Predicting user preferences in a music service

The project develops an end-to-end pipeline for processing big data and building a machine learning model that predicts whether a user will like a music track

Introduction

Overview

Music streaming platforms produce a constant flow of behavioral data: plays, replays, skips, selections, and session-level interactions. The technical challenge is not only storing those events, but turning them into something useful quickly and at scale. A recommender has only a short moment to decide what to surface next, so the supporting data pipeline has to be reproducible, scalable, and analytically trustworthy.

This project builds an end-to-end big data workflow for that setting. Starting from raw user-track interaction logs and Spotify-style track metadata, we create a pipeline that loads the data into PostgreSQL, ingests it into HDFS, prepares it for analytics in Hive, explores it through EDA, and then trains distributed Spark ML models to estimate whether a user is likely to engage positively with a track.

The implementation uses a standard Hadoop ecosystem stack:

- PostgreSQL for relational storage
- Sqoop for relational-to-HDFS ingestion
- Hive for the warehouse and SQL analytics layer
- Spark on YARN for predictive analytics
- Apache Superset for the final presentation layer

Business Objectives

The business question behind the project is simple: given a user and a track, can we estimate whether the user is likely to respond positively?

That question matters for several practical reasons:

- **Recommendation quality.** If the system can score candidate tracks before ranking them, it can surface more relevant music and reduce low-value recommendations.
- **Retention.** Better next-track decisions typically mean longer sessions and better user satisfaction.
- **Automation.** A preference model can support autoplay, recommendation rails, and playlist generation more effectively than simple heuristics alone.
- **Content understanding.** The analysis can also reveal which audio characteristics are associated with positive engagement, which is useful for editorial strategy and cold-start decisions.

The current implementation frames Stage III as recommendation-style soft binary classification:

```
label = interaction_flag
rel_score = P(interaction_flag = 1)
```

Here, `label` is the supervised binary target, while `rel_score` is the model-generated soft recommendation score. This score is later used both for thresholded prediction and for ranking-oriented offline evaluation.

For evaluation, the main optimization metric is Area Under the Precision-Recall Curve (AUC-PR), with AUC-ROC, accuracy, precision, recall, and F1 used as supporting metrics. Because the recommendation use case is ranking-oriented, top-k ranking metrics are also reported on the sampled test candidate set.

Data Description

Where the Data Comes From

The dataset comes from Kaggle: `huynguyen1902/music-interaction` . It combines:

- user-track interaction logs
- track metadata

- audio features

The repository downloads four CSV files through the Kaggle CLI:

- `tracks_features.csv`
- `interaction.csv`
- `train.csv`
- `test.csv`

In the implemented pipeline, only `tracks_features.csv` and `interaction.csv` are loaded into PostgreSQL and carried through HDFS, Hive, and Spark. The provided `train.csv` and `test.csv` files are part of the original dataset bundle, but the current Stage III pipeline builds its own user-disjoint train, validation, and test splits directly from Hive data.

Data Characteristics

The core analytical data in the project consists of two main tables, `tracks` and `interactions`, with a combined raw size of roughly 0.9 GB. The full downloaded dataset bundle is larger because it also contains the auxiliary `train.csv` and `test.csv` files.

File	Size	Rows	Role in the Project
`tracks_features.csv`	330 MB	1,204,025	Main track metadata and audio feature table
`interaction.csv`	574 MB	11,808,554	Main user-track interaction log
`train.csv`	257 MB	626,141	Downloaded from source dataset, not used in the current ML pipeline
`test.csv`	91 MB	200,575	Downloaded from source dataset, not used in the current ML pipeline

The dataset includes a temporal feature, `ts`, which stores the Unix timestamp of each interaction. In PostgreSQL it is stored as `BIGINT`. In Stage III it is converted into time-derived features such as year, month, weekday, and hour. The raw timestamp is not fed into the model directly as a plain numeric feature.

Feature Description

The `tracks` table stores one row per track and contains both metadata and audio descriptors. Important columns include:

- identifiers: `id`, `album_id`
- descriptive metadata: `name`, `album`, `artists`, `artist_ids`
- structural metadata: `track_number`, `disc_number`, `explicit`, `year`, `release_date`
- audio features: `danceability`, `energy`, `key`, `loudness`, `mode`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence`, `tempo`, `duration_ms`, `time_signature`

The `interactions` table stores user events:

- `id`: auto-generated interaction ID
- `user_id`: anonymous user identifier
- `item_id`: track identifier linked to `tracks.id`
- `interaction_flag`: binary outcome used as the training label
- `ts`: event timestamp

Architecture of Data Pipeline

The pipeline is organized as four stages. Each stage takes the output of the previous one and produces reusable artifacts for the next layer of the workflow.

Stage	Main Input	Main Output
Stage I	Raw CSV files from Kaggle	PostgreSQL tables, AVRO+Snappy files in HDFS, `.avsc` and `.java` schema artifacts

Stage	Main Input	Main Output
Stage II	Sqoop-generated AVRO files in HDFS	External Hive tables, optimized ORC+Snappy Hive tables, EDA result tables, exported CSV summaries
Stage III	Partitioned and bucketed Hive tables	Trained Spark ML pipelines, prediction files, evaluation table, generated train/test feature exports
Stage IV	Stage II and Stage III analytical outputs	Superset-ready Hive tables for Stage III outputs, published dashboard, final charts and storytelling layer

The Input and the Output for Each Stage

Stage I

- **Input:** `tracks_features.csv` , `interaction.csv`
- **Output:** PostgreSQL tables `tracks` and `interactions` ; Sqoop-imported AVRO+Snappy files in HDFS; `output/tracks.avsc` , `output/interactions.avsc` , and generated Java schema files

Stage II

- **Input:** Sqoop-imported AVRO files and AVRO schemas
- **Output:** Hive database `team11_projectdb` ; optimized Hive tables `tracks_part` and `interactions_part` ; EDA result tables `q1_results` to `q6_results` ; exported files `output/q1.csv` to `output/q6.csv` ; execution logs such as `output/hive_results.txt` and `output/beeline_*.stderr`

Stage III

- **Input:** Hive tables `team11_projectdb.tracks_part` and `team11_projectdb.interactions_part`
- **Output:** trained models in `models/model1` and `models/model2` ; prediction CSV files; `output/evaluation.csv` ; generated train/test JSON exports;

Stage IV

- **Input:** EDA outputs, model outputs, dashboard-ready tables and charts
- **Output:** Hive external tables over Stage III CSV outputs, Superset dashboard, dashboard screenshot, and published dashboard URL

Data Preparation

Data Collection

Data collection is automated through `scripts/data_collection.sh` . The script downloads the Kaggle dataset into the local `data/` directory and checks that all required files are present before Stage I continues. To keep the stage reproducible and practical on a shared cluster, the script skips the download if the files already exist locally.

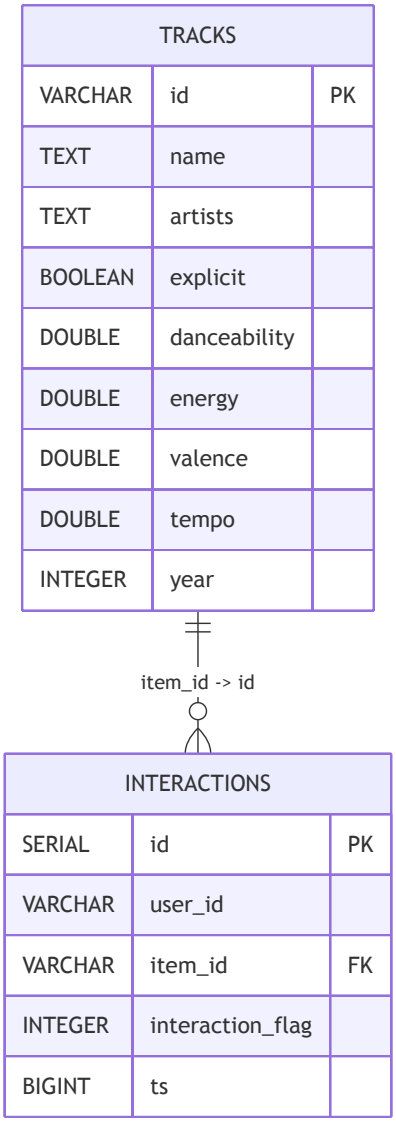
That behavior matters because the dataset is large enough that repeated downloads would waste both time and bandwidth without improving the workflow.

ER Diagram

The relational model is intentionally small and clean. The project uses two core tables:

- `tracks` , which stores one row per track and all available track-level metadata and audio features
- `interactions` , which stores user events and links each event to a track through `item_id -> tracks.id`

The relationship is one-to-many: one track can appear in many user interactions.



In this schema, `tracks.id` is the content key, while `interactions.item_id` is a foreign key that connects each event to a known track. The `interactions` table also has indexes on `user_id`, `item_id`, and `ts`, which support later joins and time-based processing.

Some Samples from the Database

After loading, the PostgreSQL database contains:

- `tracks`: 1,204,025 rows
- `interactions`: 11,808,554 rows

The verification SQL also checks:

- a small sample of rows from both tables
- non-null counts for selected track columns
- whether orphan interaction rows exist after foreign-key enforcement
- the number of distinct users and distinct tracks in the interaction log
- the minimum and maximum timestamp values in `interactions`

Representative verification queries include:

```
SELECT COUNT(*) FROM tracks;
SELECT COUNT(*) FROM interactions;
SELECT * FROM tracks LIMIT 5;
SELECT * FROM interactions LIMIT 5;
```

Loading Data into PostgreSQL

PostgreSQL is built through `scripts/build_projectdb.py`, which runs three SQL files in sequence:

1. `sql/create_tables.sql`
2. `sql/import_data.sql`
3. `sql/test_database.sql`

The first script drops and recreates the tables so that the stage can be rerun cleanly. The second loads CSV data through `COPY ... FROM STDIN`, which is appropriate for large flat files because it is both faster and simpler than row-by-row inserts. The third script runs validation queries to confirm that the load completed correctly.

The schema includes:

- primary key on `tracks.id`
- primary key on `interactions.id`
- foreign key from `interactions.item_id` to `tracks.id`
- indexes on `interactions.user_id`, `interactions.item_id`, and `interactions.ts`

These design choices matter later because Stages II and III rely heavily on joins between tracks and interactions and on timestamp-aware transformations.

Ingestion into HDFS via Sqoop

Once the relational tables are loaded, Stage I imports them into HDFS through Sqoop using `import-all-tables`. The implemented configuration stores the output in AVRO format and compresses it with Snappy:

```
sqoop import-all-tables \
  --connect jdbc:postgresql://hadoop-04.uni.innopolis.ru/team11_projectdb \
  --username team11 \
  --password $password \
  --compression-codec=snappy \
  --compress \
  --as-avrodatafile \
  --warehouse-dir=project/warehouse \
  --m 1
```

Before importing, the HDFS warehouse directory is removed so that leftover files from earlier runs do not contaminate the next execution. Sqoop also generates `.avsc` and `.java` artifacts, which are copied into the repository `output/` directory and reused in Stage II when Hive external tables are defined.

Why AVRO and Snappy in Stage I?

Stage I is primarily an ingestion stage, so AVRO is a practical choice: it is row-oriented, compact, and easy to connect to Hive through external schemas. Since Sqoop already generates the AVRO schemas during import, AVRO fits naturally into the next stage of the pipeline.

Snappy was chosen because it gives a good speed-to-compression tradeoff on a shared cluster. It does not compress as aggressively as Gzip, but it is faster to compress and decompress, which makes repeated development runs more practical.

Creating Hive Tables and Preparing the Data for Analysis

Stage II begins by copying the AVRO schema files from the repository into HDFS under `project/warehouse/avsc`. The main Hive build is driven by `scripts/stage2.sh`, which:

- uploads the AVRO schemas to HDFS
- clears the previous Hive warehouse directory in HDFS
- runs `sql/db.hql` through beeline
- executes `q1.hql` to `q6.hql`

- exports each result table into `output/qN.csv`
- stores beeline stderr logs for debugging and reproducibility

The first layer in Hive consists of two external tables built directly on top of the Stage I Sqoop output:

- `tracks`
- `interactions`

These are AVRO-backed external tables used as the staging layer. After that, Stage II creates optimized analytical tables:

- `tracks_part` : partitioned by `year` , bucketed by `id` into 11 buckets, stored as ORC with Snappy compression
- `interactions_part` : partitioned by `interaction_flag` , bucketed by `user_id` into 11 buckets, stored as ORC with Snappy compression

This is an important architectural distinction:

- **Stage I storage** is optimized for ingestion and schema handoff (`AVRO + Snappy`)
- **Stage II storage** is optimized for analytics (`ORC + Snappy`)

That split matches the actual usage pattern of the pipeline. AVRO works well as the transport format from PostgreSQL into HDFS, while ORC is a stronger choice for repeated analytical reads in Hive and downstream consumption by Spark.

The partitioning and bucketing choices also reflect how the data is used:

- partitioning `tracks_part` by `year` supports time-aware slicing of the catalog
- partitioning `interactions_part` by `interaction_flag` supports quick class-aware inspection and aligns with Stage III's target definition
- bucketing the two tables adds more structure for repeated joins and scans

After the optimized tables are materialized, the original unoptimized external Hive tables are dropped from the metastore. Because they were created as external tables, dropping them removes only metadata and keeps the underlying HDFS data intact.

Stage II also enables dynamic partitioning, enforces bucketing, switches Hive execution to Tez, and enables output compression with Snappy.

Data Analysis

The EDA part of Stage II is organized around six query files:

- `q1.hql`
- `q2.hql`
- `q3.hql`
- `q4.hql`
- `q5.hql`
- `q6.hql`

Each query:

1. creates a result table in Hive
2. materializes the result into HDFS
3. exports a CSV summary into the local `output/` directory

These six analyses are not just descriptive summaries. They are chosen to guide later modeling and business interpretation:

- class balance of the dataset
- artist loyalty structure
- audio profile differences between positive and negative rows
- popularity/exposure effects
- user activity concentration
- the gap between overall positive rate, cold-catalog behavior, and hit-track behavior

Analysis Results

Insight 1: Class Balance of Positive and Negative Rows

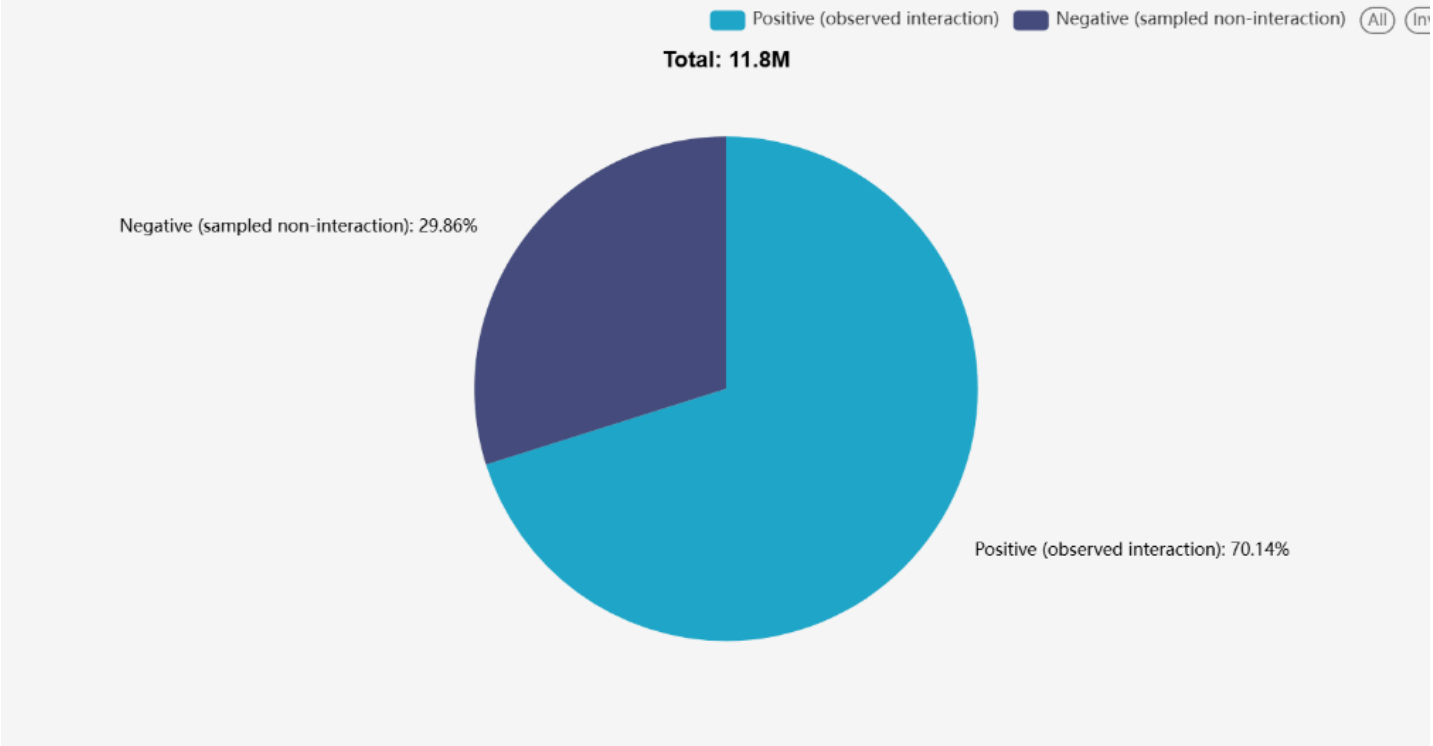
The first query measures how many interactions belong to each class:

interaction_flag	count	percent
0	3,525,682	29.8570%
1	8,282,872	70.1430%

Insight. Roughly 7 in 10 rows are positive listens and 3 in 10 are negative rows included so the model can learn contrast.

Why stakeholders should care. This should not be read as "70% of users love everything." It says something more technical and more useful: the dataset is explicitly set up for prediction, with both positive and negative examples present. That means training and reporting have to use metrics that work with imbalance. Naive accuracy alone is not enough.

Decision implication. When evaluating a recommender, the team should focus on score quality and ranking quality, not only on a raw "percent correct" figure.



Insight 2: Artist Loyalty Exists, but Mostly as One-Off Engagement

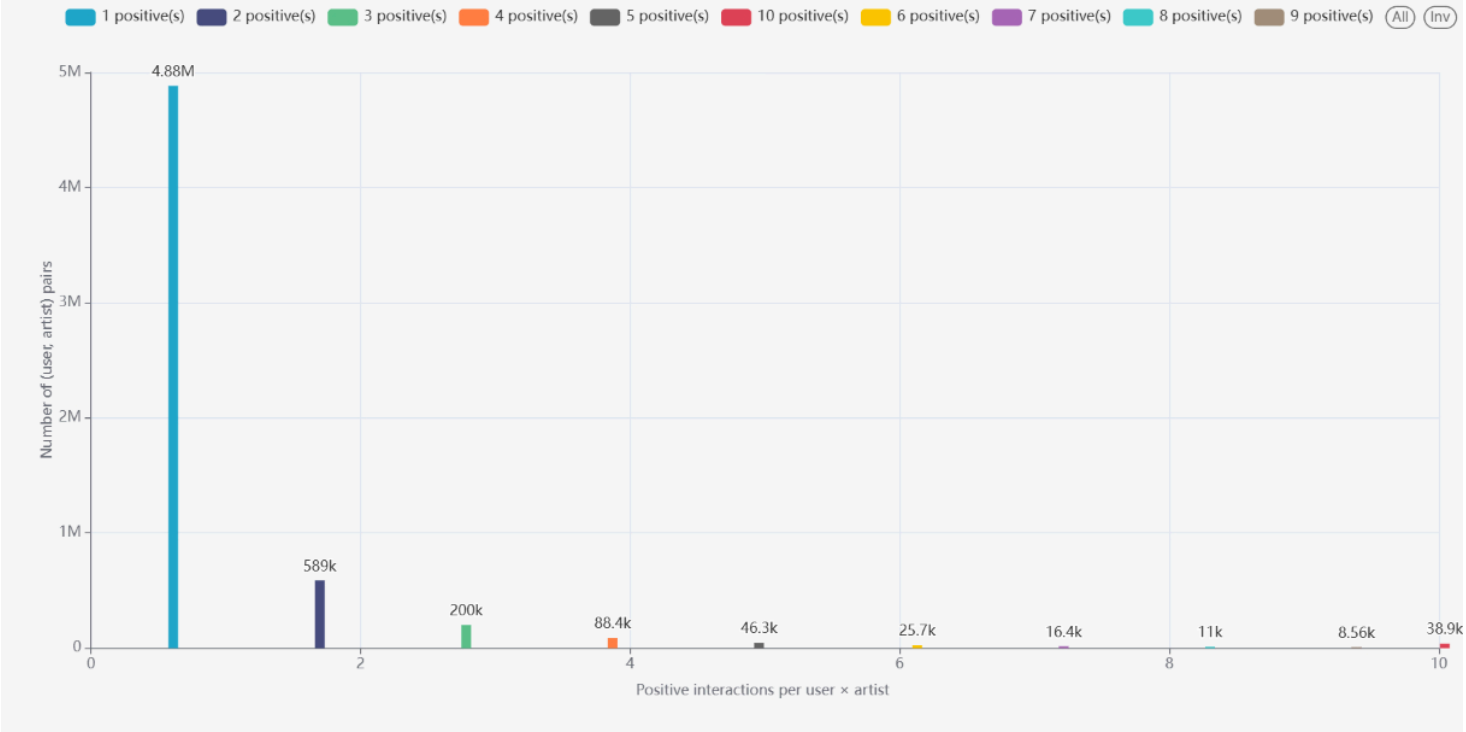
The second query studies how many positive interactions each `(user, artist)` pair receives.

The strongest result is that 82.65% of positive user-artist pairs occur exactly once.

Insight. For most user-artist pairs there is only one positive interaction; repeat engagement with the same artist is the exception rather than the norm.

Why stakeholders should care. "Because you liked Artist X, try more Artist X" is not a universal recommendation strategy. It only fits a minority of user-artist relationships. Much of music discovery appears to be shallow or one-off rather than built on deep artist loyalty.

Decision implication. The product should invest not only in artist-centric surfaces, but also in track-level, mood-level, and discovery-oriented recommendation logic.



Insight 3: Positive Rows Skew Toward More Energetic and More Danceable Tracks

The third query compares average audio features across the two classes:

interaction_flag	n	avg_danceability	avg_energy	avg_valence	avg_acousticness	avg_loudness	avg_tempo
0	3,525,682	0.4931	0.5094	0.4279	0.4468	-11.8114	117.6540
1	8,282,872	0.6141	0.6430	0.5107	0.2240	-7.3167	120.9524

Insight. Positive rows skew toward more danceable, more energetic, louder, more positive-sounding tracks and away from highly acoustic tracks.

Why stakeholders should care. Engagement is not explained only by fame or popularity. How a track sounds matters. That is useful for editorial decisions, cold-start recommendations, and diversity constraints.

Decision implication. Where user history is weak, the system can lean on audio attributes such as energy, mood, and acoustic-vs-produced character instead of guessing from popularity alone.

interaction_flag	Avg	Avg	Avg	Avg	Avg	Avg	Rows
Positive (observed interaction)	0.6141	0.643	0.5107	0.224	-7.32	120.95	8.28M
Negative (sampled non-interaction)	0.4931	0.5094	0.4279	0.4468	-11.81	117.65	3.53M

Insight 4: Positive Rate Rises Sharply with Track Exposure

The fourth query groups tracks by their total interaction volume and measures the positive rate in each popularity bucket.

Selected bucket values are:

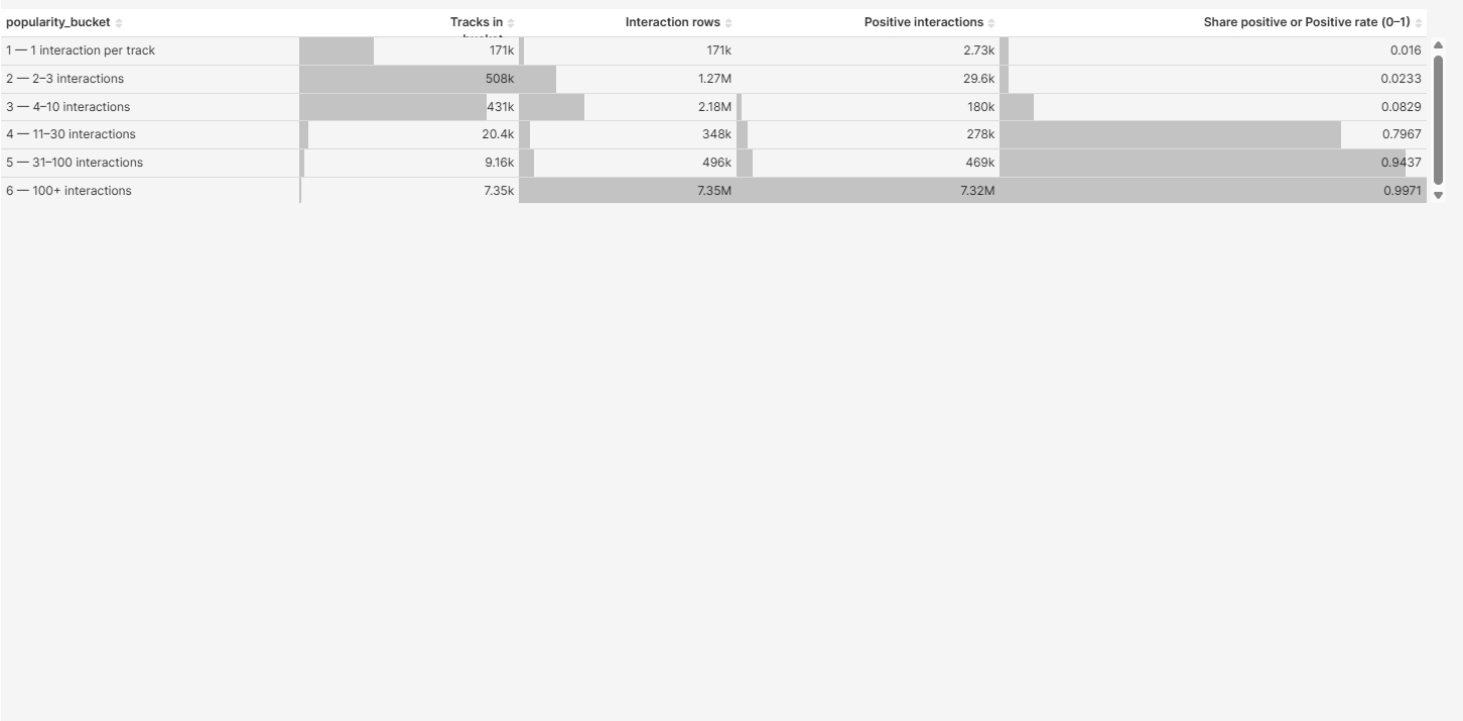
- `1` interaction: positive rate = `0.0160`
- `2–3` interactions: positive rate = `0.0233`
- `31–100` interactions: positive rate = `0.9437`
- `100+` interactions: positive rate = `0.9971`

Insight. Tracks with very few logged interactions have extremely low positive rates, while heavily exposed tracks have very high positive rates. There is a sharp jump across the middle buckets.

Why stakeholders should care. Exposure and accumulated history drive apparent satisfaction. Hits look “universally loved” partly because they are seen more, often by warm audiences; unknown tracks look much worse partly because they are cold.

Decision implication.

- Compare like-for-like segments, such as new releases versus established catalog.
- Expect cold-catalog recommendations to need different KPIs from hit-track recommendations.
- Treat popularity or exposure as a first-class feature or reporting dimension, otherwise the model risks confusing correlation with causation.



popularity_bucket	Tracks in	Interaction rows	Positive interactions	Share positive or Positive rate (0–1)
1 — 1 interaction per track	171k	171k	2.73k	0.016
2 — 2–3 interactions	508k	1.27M	29.6k	0.0233
3 — 4–10 interactions	431k	2.18M	180k	0.0829
4 — 11–30 interactions	20.4k	348k	278k	0.7967
5 — 31–100 interactions	9.16k	496k	469k	0.9437
6 — 100+ interactions	7.35k	7.35M	7.32M	0.9971

Insight 5: A Small Share of Users Produces a Large Share of Volume

The fifth query looks at how user activity is distributed.

The largest bucket is users with 6-20 interactions:

- 533,897 users
- 60.19% of all users
- 6,310,140 interactions
- 53.44% of total interaction volume

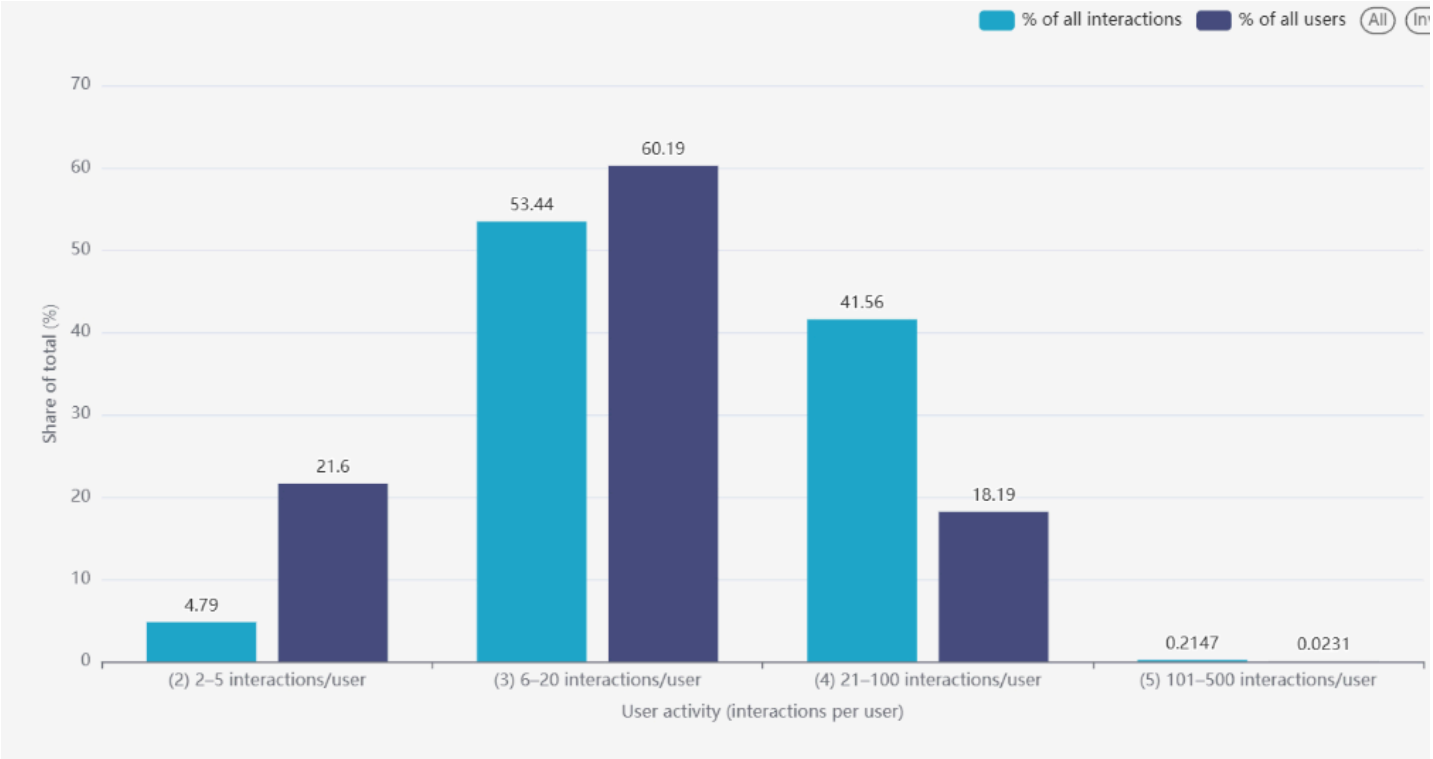
Users with 21-100 interactions account for only 18.19% of users, but 41.56% of all interactions.

Insight. A relatively small slice of users drives a large share of total interaction volume, while many users contribute only a modest amount of activity.

Why stakeholders should care. Power users can dominate metrics, dashboards, and A/B test summaries if the data is not segmented carefully.

Decision implication.

- Segment dashboards by activity tier: casual, core, and power users.
- Design retention and activation programs to move users up one tier rather than only optimizing for the heaviest listeners.
- Report experiments by segment so that one cohort does not dominate the story.



Insight 6: The Overall Positive Rate Hides Two Opposite Worlds

The sixth query compares three scenarios:

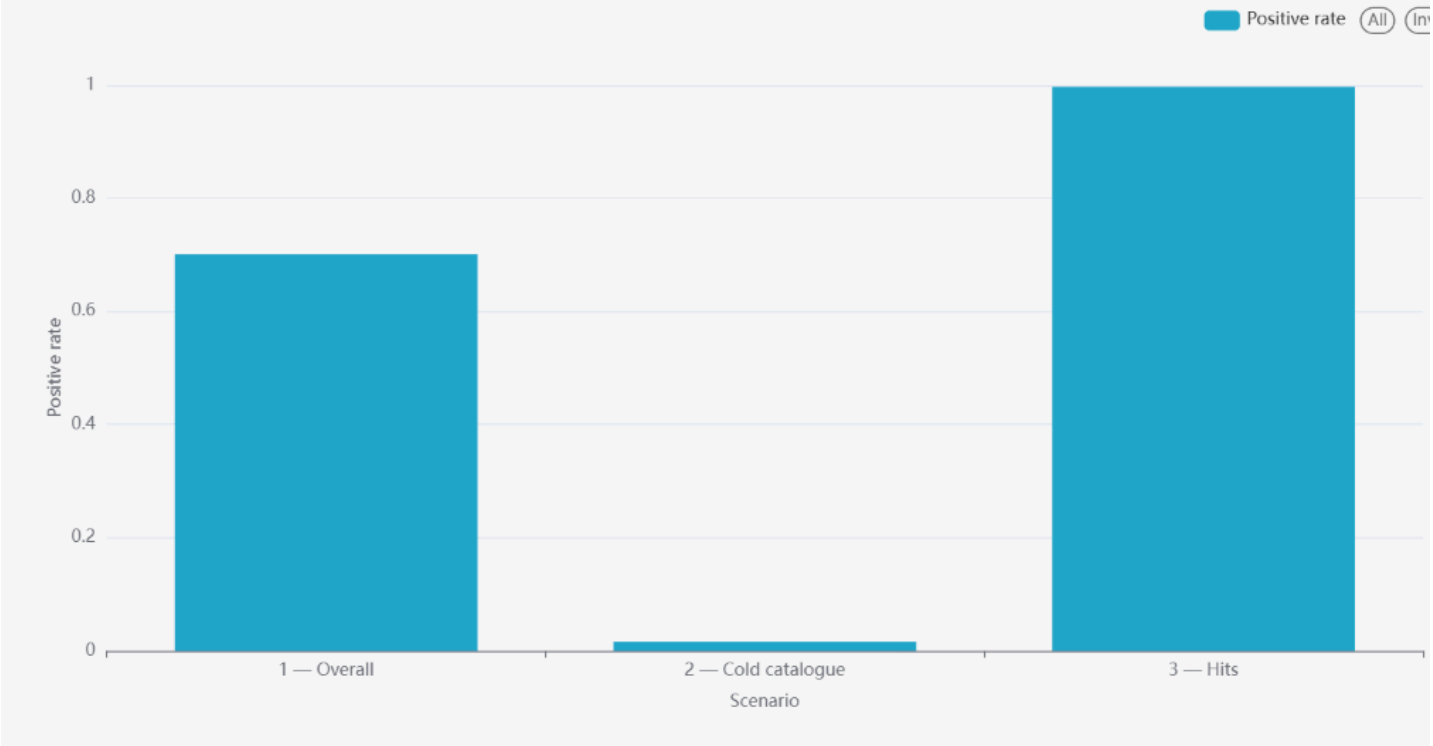
sort_key	scenario	positive_rate
1	Marginal: share of positive rows (all data)	0.7014
2	Cold tracks: 1 interaction per track	0.0160
3	Hit tracks: 100+ interactions per track	0.9971

Insight. The mixed dataset is not one homogeneous population. Cold-catalog rows are close to zero positivity per interaction, while hit tracks are almost always positive. The overall 70.14% positive rate is just a weighted average between those extremes.

Why stakeholders should care. A single headline KPI such as “about 70% of rows are positive” can be useful for model class balance, but very misleading if it is interpreted as uniform user satisfaction or uniform recommendation difficulty.

Decision implication.

- Split reporting and targets by cold versus established inventory tiers.
- Treat popularity and exposure as a first-class modeling and dashboard dimension, not something averaged away.



Charts

The six charts above provide the visual backbone of the EDA stage. Together, they show that:

- the dataset is intentionally structured for predictive learning
- user-artist repetition exists but is usually shallow
- track sound characteristics matter for engagement
- popularity strongly shapes the observed label distribution
- user activity is unevenly distributed
- the overall positive rate hides major differences between cold and hit inventory

In other words, the visual analysis does more than decorate the report. It sets up the modeling assumptions used in Stage III and gives stakeholders a clearer explanation of what the data is actually saying.

Interpretation

The main interpretive lesson from Stage II is that user preference in this dataset is not a single, uniform phenomenon. It is shaped by:

- track content
- user history
- exposure/popularity
- activity concentration

That matters because each of those forces can affect both business decisions and model performance. A recommender that ignores these structural differences can look good on aggregate while still behaving poorly on cold inventory, casual users, or discovery use cases.

ML Modeling

Stage III uses Spark ML on YARN and reads only the optimized Hive tables produced in Stage II. No local CSV files are used for model training. This is important because it keeps the predictive stage aligned with the project requirement that distributed processing should happen on the cluster, not on a local machine.

Although the broader product goal is recommendation, the implemented Stage III pipeline frames the problem as recommendation-style soft binary preference classification:

```
label = interaction_flag
rel_score = P(interaction_flag = 1)
```

Here, `rel_score` is the soft recommendation score that supports both thresholded prediction and ranking-oriented offline evaluation.

Feature Extraction and Data Preprocessing

The pipeline reads:

- `team11_projectdb.interactions_part`
- `team11_projectdb.tracks_part`

The two tables are joined through track ID, and the supervised target is:

```
label = interaction_flag
```

The split is user-disjoint:

- train users: hash bucket `< 70`
- validation users: hash bucket `70–84`
- test users: hash bucket `85–99`

Inside each user history, interactions are ordered by timestamp. The first 70% of that user's interactions are used as context, and the later interactions are used as supervised target rows. This allows the model to use early user behavior to build features without leaking future labels.

Several additional safeguards are built into the pipeline:

- users with fewer than two interactions are removed
- train, validation, and test users do not overlap
- target rows exclude tracks already present in the same user's context
- decision thresholds are selected on validation rows only
- final metrics are computed on test rows only

The feature extraction pipeline is built with `pyspark.ml.Pipeline`. Its feature groups include:

- numeric track/audio fields such as `danceability`, `energy`, `valence`, `tempo`, `duration_ms`, and `year`
- categorical metadata such as `explicit`, `key`, `mode`, and `time_signature`
- artist text features via `RegexTokenizer` and `HashingTF`
- time-derived cyclic features from `ts`
- user history features from the early context rows
- user-artist history features from the early context rows

The raw Unix timestamp is not used directly as a numeric feature. Instead, it is transformed into:

- event year
- month sine/cosine
- day-of-week sine/cosine
- hour sine/cosine

Recent history features are bounded to the latest 50 context interactions:

- `prior_user_interactions`
- `prior_user_positives`
- `prior_user_positive_share`
- `prior_user_artist_interactions`

- `prior_user_artist_positives`
- `prior_user_artist_positive_share`

Missing numeric values are handled through `Imputer`. Categorical values are encoded through `StringIndexer` and `OneHotEncoder`. The transformed columns are assembled into a single Spark feature vector.

Because the train split is imbalanced, the pipeline also computes class weights using the label distribution in the training users only.

For ranking evaluation, the test set is expanded with up to 30 sampled unobserved tracks per test user. These candidates are filtered so that the user has no known interaction with the sampled track in either context or held-out target rows. They are added only for evaluation and marked with:

```
label = 0
is_sampled_candidate = 1
```

These rows are not explicit dislikes. They are offline non-relevant candidates used to make ranking evaluation less optimistic.

Training and Fine-Tuning

The implemented Stage III pipeline trains two distributed Spark ML models:

model	type
model1	`LogisticRegression`
model2	`RandomForestClassifier`

Both models are trained inside the same Spark ML feature pipeline.

Hyperparameter tuning is performed with Spark `CrossValidator` using:

- `numFolds = 3`
- `foldCol = cv_fold`
- optimization metric = `areaUnderPR`

The validation split is then used to choose the threshold for converting `rel_score` into a hard binary prediction.

The implemented search grids are:

model	hyperparameters	grid size
Logistic Regression	`regParam` x `elasticNetParam` = 3 x 2	6
Random Forest	`numTrees` x `maxDepth` = 3 x 2	6

More specifically:

- Logistic Regression searches `regParam` in `{0.001, 0.01, 0.1}` and `elasticNetParam` in `{0.0, 0.5}`
- Random Forest searches `numTrees` in `{50, 100, 150}` and `maxDepth` in `{5, 8}`

Within the implemented project scope, the predictive stage compares two distributed Spark ML scorers. Conceptually, this is still aligned with the original recommender-system goal because the output of the pipeline is not just a hard label, but a personalized engagement score `rel_score` that can be used for ranking candidate tracks.

Evaluation

Stage III writes its final comparison table to `output/evaluation.csv`. In the current run, the final evaluation set contains:

- 133,523 test users
- 4,590,102 test rows
- 502,812 true positive interactions

- 4,005,690 sampled unobserved candidate rows

The positive rate after adding sampled candidates is therefore about 10.95%. This is much harder than evaluating only on held-out labeled target rows, and it is a more realistic offline ranking setup, even though it still does not represent full-catalog recommendation.

The current model comparison is:

model	areaUnderPR	areaUnderROC	accuracy	precision	recall	f1
model1	0.4741	0.8875	0.5740	0.2014	0.9739	0.3337
model2	0.6285	0.8725	0.4861	0.1731	0.9770	0.2940

Selected hyperparameters and thresholds are:

model	best params	threshold
model1	`elasticNetParam=0.0; regParam=0.001`	0.15
model2	`maxDepth=8; numTrees=150`	0.30

Because `areaUnderPR` is the optimization metric, the best model by the main assignment criterion is **Random Forest (`model2`)**.

At the same time, the tradeoff is not one-sided. Logistic Regression performs slightly better on the top-10 ranking metrics in the current sampled-candidate evaluation:

model	precision@10	recall@10	ndcg@10	mrr@10
model1	0.3565	0.8762	0.7994	0.8373
model2	0.3423	0.8487	0.7693	0.8137

That means the final interpretation is more nuanced than “one model wins at everything.” Random Forest is the best model by the primary optimization metric, while Logistic Regression remains competitive on ranking behavior near the top of the list.

For interpretation, it is also useful to compare against a naive always-positive baseline:

baseline	accuracy	precision	recall	f1
always predict positive	0.1095	0.1095	1.0000	0.1975

Against that baseline, both trained models improve precision, accuracy, and F1 substantially, even though recall drops slightly from the trivial 100% baseline.

The report should be careful not to overclaim what the ranking metrics mean. The model is **not** evaluated over the full global track catalog. Instead, the ranking metrics are computed on each user's held-out target rows plus sampled unobserved candidates. This is a useful offline approximation, but it is not the same as ranking every track in the music catalog for each user.

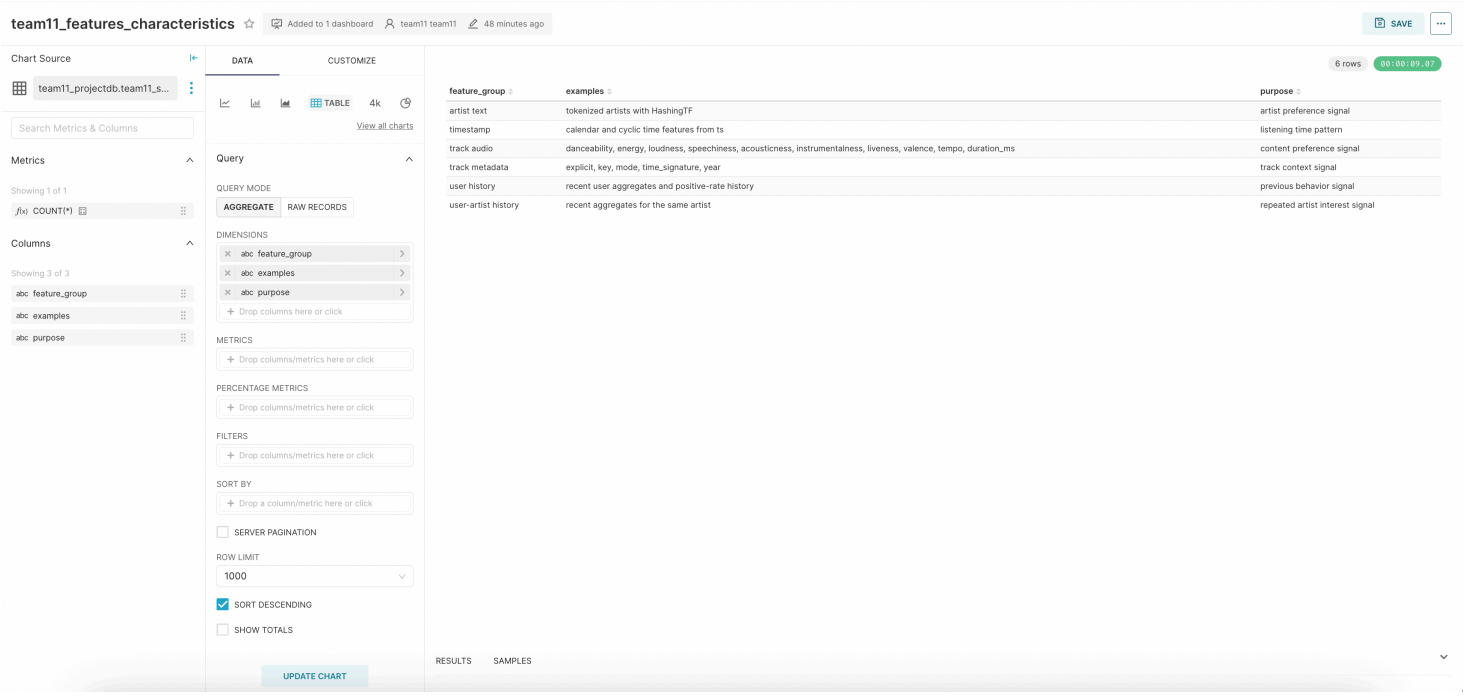
Examples of thresholded predictions can be taken directly from `output/model1_predictions.csv` :

label	prediction
0.0	0.0
1.0	1.0
0.0	1.0
1.0	1.0

The full score files with `user_id` , `item_id` , `label` , `prediction` , and `rel_score` are generated by the Stage III pipeline and can be shared separately with the large-artifact bundle if they are not kept in the Git repository.

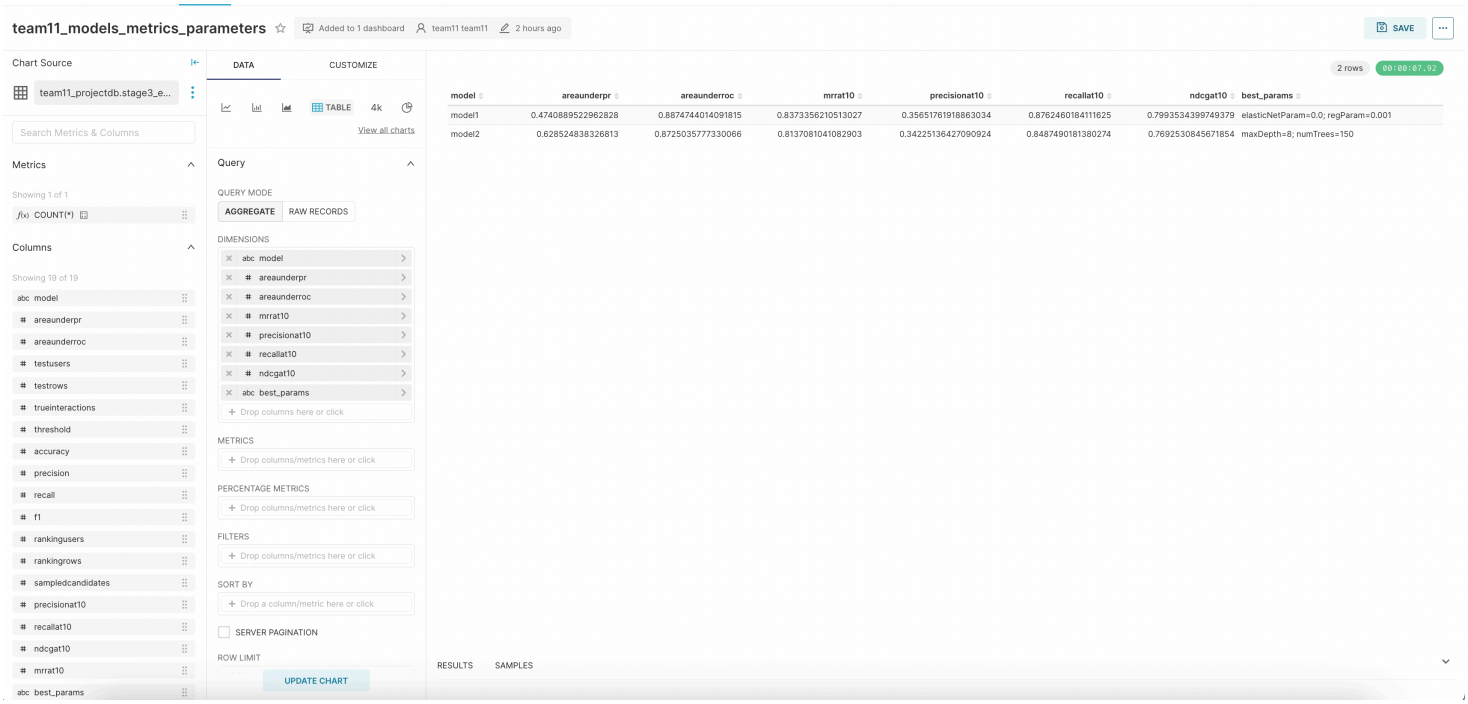
The dashboard also includes several Stage III visual panels that make the modeling behavior easier to explain to a non-technical audience.

First, the **feature-characteristics** table summarizes the six feature groups used in the final pipeline. It shows that the model is not driven by only one signal source, but combines track audio, metadata, artist text, timestamp-derived features, and short-term user history.



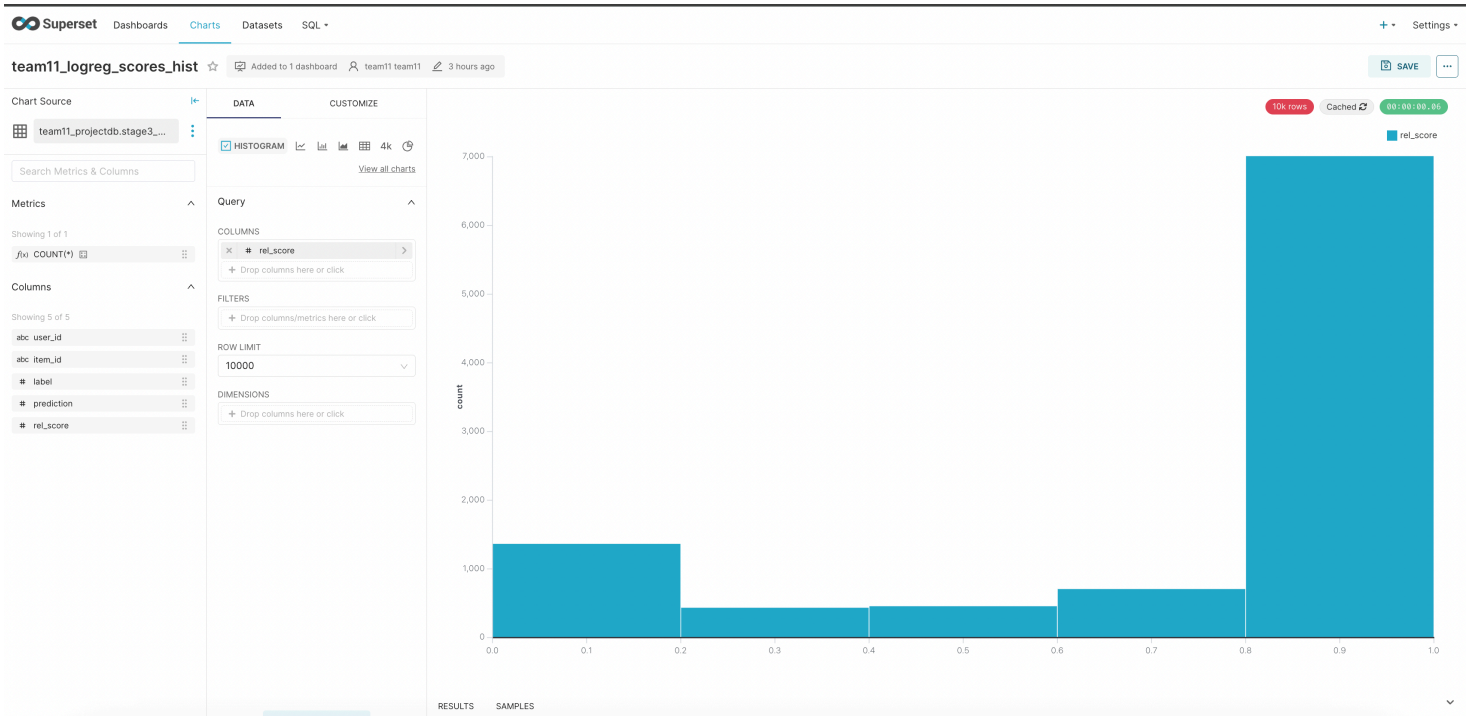
This matters because it supports the main project claim: the system is not simply memorizing popular tracks. It combines content-based and behavior-based signals, which is exactly what a practical music recommender needs in a cold-to-warm catalog setting.

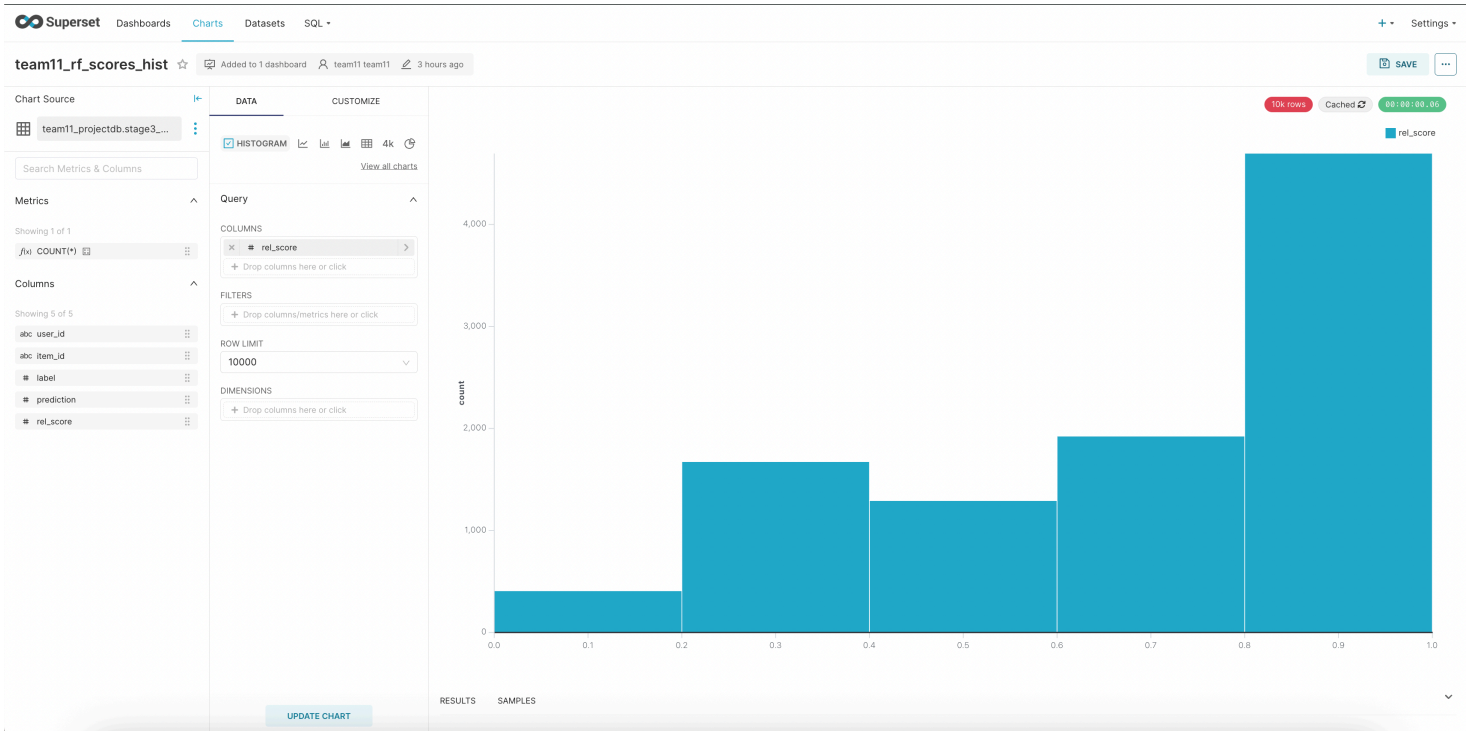
Second, the **models-metrics-parameters** table gives one compact comparison of both trained models and their selected hyperparameters.



This panel is especially useful in the defense because it shows the tradeoff cleanly. Random Forest wins on the primary optimization metric, `areaUnderPR`, while Logistic Regression keeps a small edge on some top-k ranking metrics such as `precision@10`, `ndcg@10`, and `mrr@10`. In other words, the final choice depends on whether we prioritize probability-quality under class imbalance or slightly stronger ordering near the top of the ranked list.

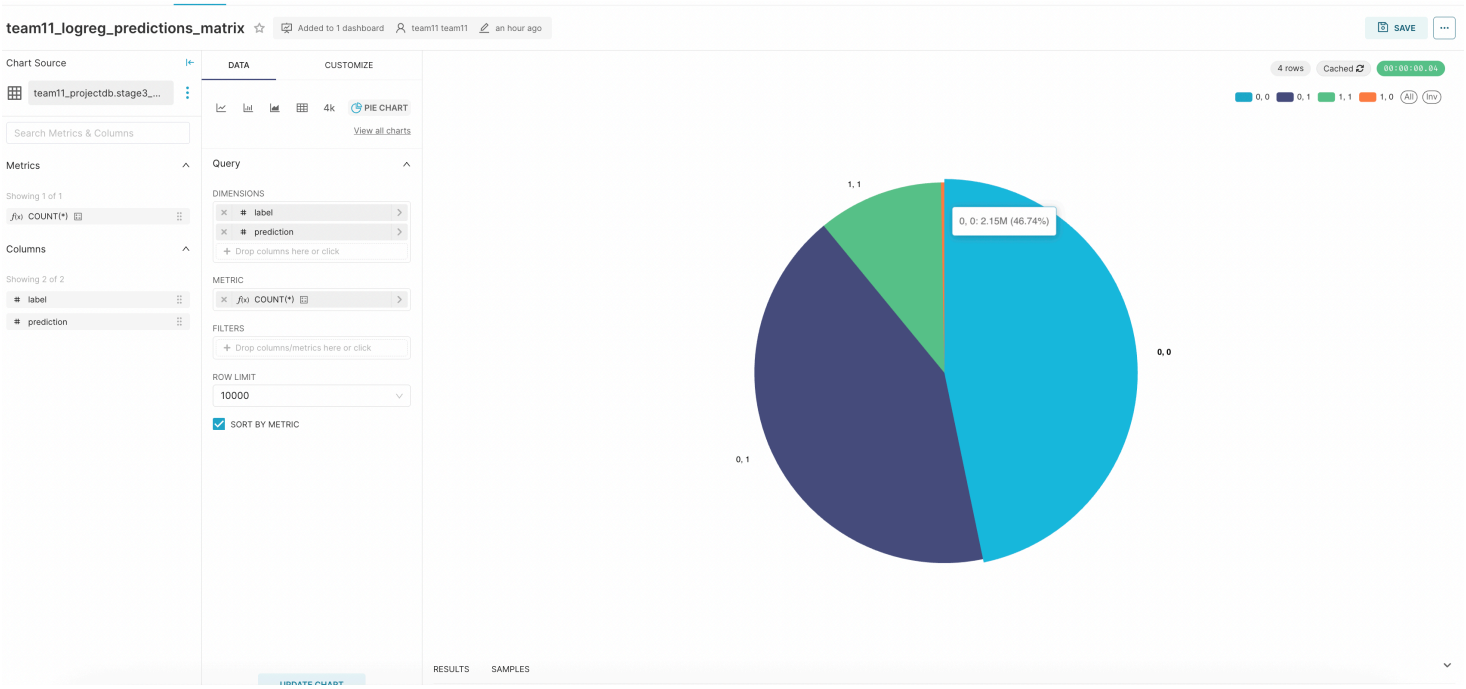
Third, the score histograms show how each model distributes its soft recommendation score `rel_score`.

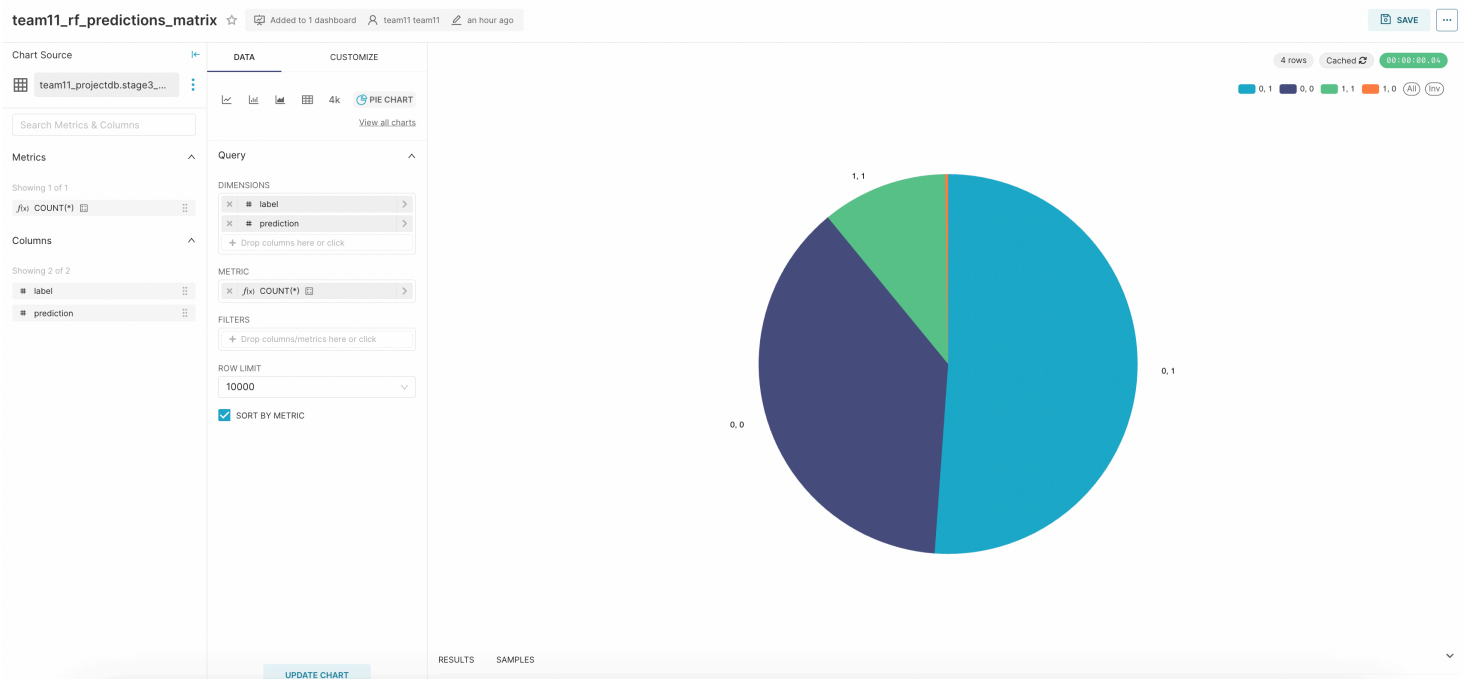




The Logistic Regression histogram is strongly polarized: a large share of rows falls near the top score bucket, and a smaller but still visible mass sits near zero, with relatively fewer rows in the middle. This means the model tends to make more decisive score assignments. The Random Forest histogram is also skewed toward high scores, but it spreads more mass through the middle buckets. That pattern is consistent with the final metrics: Random Forest improves overall positive-class retrieval quality, but Logistic Regression stays slightly sharper in top-of-list ranking.

Finally, the prediction-matrix charts summarize the thresholded outputs of both models by `(label, prediction)` pair.





These charts make the error pattern visually obvious. In both models, true positives remain high and false negatives are very small, which matches the very high recall values. At the same time, false positives occupy a large share of the output space, which explains the modest precision values. This is the expected tradeoff in an imbalanced recommendation-style setting where the threshold is chosen to preserve recall and ranking usefulness rather than to maximize strict binary precision alone.

Data Presentation

The final presentation layer is designed as a single Apache Superset dashboard that brings together the descriptive, analytical, and predictive parts of the project. The goal is not just to place several charts on one screen, but to tell a coherent story about:

- what data the pipeline stores
- what patterns emerge from user behavior
- how well the predictive model captures those patterns

The dashboard is published in Apache Superset and is available at:

Music recommendation modeling dashboard

As allowed by the course instructions, the dashboard itself is the one part of the project that is assembled manually inside Superset rather than through shell or Python automation. The implemented repository still prepares the inputs for that stage. In particular, `scripts/stage4.sh` and `sql/stage4.hql` register the Stage III outputs in Hive as external tables:

- `stage3_evaluation`
- `stage3_model1_predictions`
- `stage3_model2_predictions`
- `stage3_model1_scores`
- `stage3_model2_scores`

This makes the dashboard reproducible at the data-preparation level even though the visual assembly inside Superset remains manual.

Description of Each Chart

The final dashboard should include at least the following charts and panels.

Chart 1: Class Balance of `interaction_flag`

Shows how the interaction data is divided between positive and negative rows. This establishes the class structure of the prediction problem and motivates the use of class-sensitive metrics.

Chart 2: Positive Interactions per User-Artist Pair

Shows how often users repeatedly engage positively with the same artist. This supports the decision to use artist-aware history features without assuming deep artist loyalty everywhere.

Chart 3: Average Audio Features by Interaction Class

Compares the average audio characteristics of positive and negative rows. This shows that the audio attributes are genuinely informative and not only decorative metadata.

Chart 4: Positive Rate by Track Popularity Bucket

Shows how positive outcomes vary by exposure and track popularity. This makes the popularity bias visible and warns against overly simple KPI interpretation.

Chart 5: User Activity Distribution

Shows how interaction volume is distributed across user-activity tiers. This justifies the user-disjoint split strategy used in Stage III.

Chart 6: Overall vs Cold vs Hit Positive Rate

Shows that the global positive rate is only a weighted average across very different inventory tiers. This is important for both business interpretation and model design.

Chart 7: `team11_features_characteristics`

Summarizes the feature pipeline in business language. The table groups the final features into artist text, timestamp, track audio, track metadata, user history, and user-artist history. This is useful because it explains what information the model actually sees instead of presenting the ML stage as a black box.

Chart 8: `team11_models_metrics_parameters`

Compares the two trained models side by side using both classification and ranking-aware metrics, and shows their selected hyperparameters. This is the main evidence panel for why Random Forest is the best model by `areaUnderPR`, while Logistic Regression still remains competitive

on top-k ranking quality.

Chart 9: `team11_logreg_scores_hist`

Shows the distribution of `rel_score` for Logistic Regression. The chart suggests a fairly polarized scorer, with many rows near the highest score bucket and a smaller low-score mass near zero. This supports the interpretation that Logistic Regression behaves like a sharper scorer near the ranking boundary.

Chart 10: `team11_rf_scores_hist`

Shows the distribution of `rel_score` for Random Forest. The score mass is still concentrated toward the positive end, but it is more spread through the middle than in Logistic Regression. This is consistent with a model that retrieves positives strongly overall while being slightly less aggressive at the very top of the ranked list.

Chart 11: `team11_logreg_predictions_matrix`

Shows the thresholded `(label, prediction)` distribution for Logistic Regression. The chart makes it easy to see that the model preserves very high recall with very few false negatives, but also produces many false positives, which is why precision stays much lower than recall.

Chart 12: `team11_rf_predictions_matrix`

Shows the thresholded `(label, prediction)` distribution for Random Forest. The same high-recall pattern appears here, but with an even larger false-positive share than Logistic Regression. This aligns with the lower precision and lower accuracy of Random Forest despite its better `areaUnderPR`.

Chart 13: Example Predictions

Shows a few concrete prediction outputs including `label`, `prediction`, and, when needed, `rel_score`. This final panel is useful during the defense because it brings the discussion back from aggregate metrics to individual rows that stakeholders can understand directly.

Your Findings

Several consistent findings emerge from the pipeline.

First, user preference in this dataset is shaped by both content and interaction context. Positive rows are associated with tracks that are more energetic, more danceable, louder, less acoustic, and somewhat more positive in mood.

Second, user behavior is uneven. A relatively small share of users contributes a large share of the interaction volume. That makes segment-aware reporting and user-disjoint evaluation important.

Third, popularity strongly affects the observed label distribution. Highly exposed tracks behave very differently from cold-catalog tracks, which means that aggregate positive rate alone can be misleading.

Fourth, the predictive stage confirms that the engineered feature space captures useful signal. Both distributed Spark models beat a naive always-positive baseline. Random Forest achieves the best result by the primary optimization metric, while Logistic Regression remains slightly stronger on some top-10 ranking measures.

Fifth, the ML charts make the model behavior easier to interpret. The feature summary confirms that the model combines content, context, and history instead of relying on one narrow proxy. The score histograms show that both models assign many rows high engagement probability, but Logistic Regression is more sharply polarized while Random Forest spreads more probability mass through the middle. The prediction matrices show why recall is so high and precision remains modest: the pipeline is intentionally tolerant of false positives because ranking usefulness matters more than conservative binary filtering.

Most importantly for the framing of the project, the predictive component should not be thought of as “just classification in disguise.” The pipeline produces a soft recommendation score and evaluates that score with ranking-aware metrics on a sampled candidate set. In other words, the training objective is supervised binary preference prediction, but the product interpretation is recommendation-style scoring.

Taken together, these findings suggest that a useful recommender in this domain should combine:

- track-level content understanding
- user and user-artist history
- careful treatment of popularity and exposure
- evaluation protocols that reflect ranking rather than only raw classification accuracy

Conclusion

This project demonstrates an end-to-end big data workflow for music preference prediction. Starting from raw CSV files, the pipeline loads data into PostgreSQL, ingests it into HDFS with Sqoop, organizes it in Hive for analytics, explores it through EDA, and then trains distributed Spark ML models on YARN.

The analytical results show that positive interactions are not random. They correlate with track audio characteristics, with user-listening structure, and with strong popularity effects. The ML stage confirms that these signals are predictive: after expanding the test set with sampled unobserved candidates, both models still outperform a naive baseline. Random Forest achieves the highest AUC-PR, while Logistic Regression remains competitive on top-10 ranking quality.

The remaining work for the final submission is mainly presentation-layer work: finalizing the Superset dashboard, inserting the final dashboard screenshots or public link, and completing the course-specific administrative sections.

Summary of the Report

The project builds a scalable recommendation-oriented data pipeline for predicting whether a user will engage positively with a music track. The workflow begins with Kaggle data, stores it in PostgreSQL, ingests it into HDFS, prepares it in Hive, analyzes it through EDA, and finally trains Spark ML models on YARN.

Stage II shows that the dataset is imbalanced, that most positive user-artist relationships are one-off rather than deep, that positive rows correlate with a distinct audio profile, and that exposure strongly shapes apparent satisfaction. Stage III turns those insights into a distributed feature pipeline and compares two Spark ML models on an offline ranking-oriented test setup with sampled candidates.

Random Forest is the best model by the primary optimization metric, while Logistic Regression remains slightly stronger on some top-k ranking metrics. The core analytical and predictive parts of the project are therefore in place, with the main remaining work concentrated in the final dashboard and presentation materials.

Reflections on Own Work

One strength of the project is that it was built as a connected system rather than as a set of isolated scripts. The storage decisions in Stage I support Stage II naturally, and the Hive design in Stage II supports the Spark ML workflow in Stage III without requiring a separate local preprocessing path.

Another strength is that the EDA stage was used to inform modeling decisions rather than being treated as a decorative requirement. Class balance, user activity distribution, popularity bias, and audio-feature differences all influenced the predictive design.

As a team, we also learned a useful practical lesson about project framing. We started from a recommender-system idea, but the course requirements were expressed through Spark ML supervised models. Instead of pretending those are unrelated worlds, we had to connect them: use a supervised preference target, output a soft score, evaluate with ranking-aware metrics, and explain clearly where the current pipeline is recommendation-style scoring and where it is still only an offline approximation. That clarification made both the code and the report stronger.

The project moved more smoothly once responsibilities became stage-oriented: one teammate could focus on ingestion and repository structure, another on Hive and EDA, another on Spark ML, and another on dashboarding and report integration. At the same time, we still had to spend real time on syncs between stages and during active development: explaining schema choices, handing over outputs, checking whether downstream scripts still matched upstream changes, and aligning the technical story with the final report. That coordination work is easy to underestimate, but in practice it was one of the reasons the project stayed coherent.

Challenges and Difficulties

The first challenge of the project is orchestration across several systems. Even with a relatively simple schema, the workflow crosses PostgreSQL, HDFS, Hive, Spark, and Superset. Small mismatches in schema handling, HDFS paths, or cleanup logic can break downstream stages 😃

The second challenge is reproducibility. Because the project is tested through rerunnable scripts, each stage has to clean up or overwrite its previous outputs carefully. This requires discipline in dropping objects, clearing warehouse directories, and handling existing artifacts safely.

The third challenge is interpretation. The dataset is large and rich, but it is not behaviorally homogeneous. Popularity effects are strong, and ranking evaluation is sensitive to how candidate sets are constructed. The sampled-candidate setup in Stage III is a useful improvement over target-only evaluation, but it still remains an offline approximation.

The final challenge is communication. A technically correct pipeline is not enough on its own. The results also have to be presented in a way that helps business stakeholders understand what is signal, what is bias, and where the product decisions should change.

Recommendations

Several improvements would strengthen the final version of the project.

- Keep popularity or exposure as a first-class reporting dimension instead of relying only on one global positive-rate KPI.
- Attach large Stage III artifacts through a clearly labeled external folder so evaluators can inspect generated train/test exports, models, and score files if needed.
- Extend the final dashboard with a compact feature-summary panel and a model-output panel so the audience sees both aggregate metrics and concrete predictions.
- If the final defense discussion pushes more strongly toward full recommendation framing, the next iteration should add a broader candidate-generation layer and possibly another model family optimized directly for ranking.

From a product perspective, the next logical step after this project would be to add a broader candidate-generation layer and evaluate ranking quality on a richer candidate pool, rather than only on held-out target rows plus sampled offline candidates.

Table of Contributions of Each Team Member

Project Task	Task Description	Kseniya Korchagina	Mariia Chugaeva	Danis Sharafiev	Diana Minnakhmetova	Deliverables	Average Hours Spent
Repository setup and template preparation	Initialize the project repository, adapt the template, structure folders, and prepare the base workflow	75%	0%	25%	0%	repository skeleton, initial README, project structure	3.0
Stage I data extraction and PostgreSQL ingestion	Download the dataset, define relational schema, load CSV files into PostgreSQL, and prepare	50%	15%	25%	10%	`scripts/stage1.sh` , `build_projectdb.py` , SQL schema and import scripts	7.0

Project Task	Task Description	Kseniya Korchagina	Mariia Chugaeva	Danis Sharafiev	Diana Minnakhmetova	Deliverables	Average Hours Spent
	reproducible Stage I scripts						
Stage I HDFS ingestion	Import relational tables into HDFS with Sqoop and preserve `.avsc` / `.java` artifacts	20%	10%	70%	0%	HDFS warehouse, AVRO artifacts in `output/`	5.0
Stage II Hive warehouse design	Create Hive database, build external and optimized ORC tables, add partitioning and bucketing	5%	75%	20%	0%	`scripts/stage2.sh` , `sql/db.hql` , Hive warehouse structure	8.0
Stage II EDA queries and outputs	Implement six EDA queries, export CSV summaries, and validate analytical outputs	5%	80%	15%	0%	`sql/q1.hql` ... `sql/q6.hql` , `output/q1.csv` ... `output/q6.csv`	8.0
Stage III feature engineering and ML pipeline	Build Spark ML pipeline, user-disjoint split, context-history features, and model training logic	0%	5%	95%	0%	`scripts/model.py` , `scripts/stage3.sh` , train/test exports	14.0
Stage III evaluation and metric design	Add sampled candidates, ranking-aware metrics, thresholds, and final evaluation outputs	0%	0%	100%	0%	`output/evaluation.csv` , prediction files, metric interpretation	12.0
Stage IV dashboard data preparation	Register Stage III outputs in Hive and prepare	0%	10%	90%	0%	`scripts/stage4.sh` , `sql/stage4.hql` , dashboard-ready Hive tables	6.0

Project Task	Task Description	Kseniya Korchagina	Mariia Chugaeva	Danis Sharafiev	Diana Minnakhmetova	Deliverables	Average Hours Spent
	datasets for Superset						
Dashboard composition and chart interpretation	Assemble Superset dashboard, organize the layout, export chart screenshots, and write chart stories	20%	35%	35%	20%	Superset dashboard, chart images, business interpretations	9.0
Report writing and presentation materials	Write the report, polish storytelling, prepare slides and speaker notes	0%	0%	0%	100%	final report, slide deck text, speaker notes	12.0
Team coordination and stage syncs	Hold working syncs, explain handoffs between stages, review assumptions, and keep the technical story aligned across code, dashboard, and report	25%	25%	25%	25%	meeting notes, handoff checks, internal reviews, aligned final narrative	4.0

Large Artifacts

Due to their size, the following Stage III artifacts are not stored in the GitHub repository.
They are available separately at the provided link:

- **Train and test feature exports** (`data/train.json` , `data/test.json`) – user-disjoint splits used for model training and evaluation.
- **Full model score files** (`output/model1_scores.csv` , `output/model2_scores.csv`) – contain `user_id` , `item_id` , `label` , `prediction` , and `rel_score` for every test row.

Download link: Google Drive

These files can be used to reproduce the evaluation metrics, inspect individual predictions,
or extend the analysis without rerunning the full pipeline.